Flight Delay Analysis using Apache Spark (PySpark)

A big data project analyzing flight delay patterns using Apache Spark (PySpark).

Authors:

Konstantinos Michaelides

Pantelis Tassopoulos

Περιβάλλον Εκτέλεσης: Google Colab (Python 3.10, Spark 3.5.0, OpenJDK 11)

1. Εισαγωγή

Στην παρούσα εργασία υλοποιήθηκε ανάλυση δεδομένων καθυστερήσεων πτήσεων με χρήση του Apache Spark, στο περιβάλλον Google Colab. Στόχος ήταν η εξαγωγή στατιστικών πληροφορίας από το dataset flights_2000.csv και η σύγκριση των υλοποιήσεων μέσω RDD API και DataFrame API ως προς απόδοση, εκφραστικότητα και δυνατότητες βελτιστοποίησης.

2. Καθαρισμός και Προεπεξεργασία Δεδομένων

Αρχικά έγινε ανάγνωση του αρχείου flights_2000.csv. Ακολούθησε καθαρισμός των δεδομένων με φιλτράρισμα εγγραφών όπου CANCELLED=0 και απομάκρυνση κενών ή μη αριθμητικών τιμών στην καθυστέρηση αναχώρησης (DEP_DELAY). Στη συνέχεια, οι τιμές μετατράπηκαν σε double τύπο για ασφαλείς υπολογισμούς.

3. Θέμα 1 — Υλοποίηση με RDD API

Στην υλοποίηση με RDD, το αρχείο διαβάστηκε με textFile() και κάθε γραμμή αναλύθηκε σε πεδία. Εξήχθησαν ζεύγη (origin_airport, delay_minutes) και υπολογίστηκε η μέση καθυστέρηση ανά αεροδρόμιο με χρήση της reduceByKey(). Τέλος, προέκυψαν τα 10 αεροδρόμια με τη μεγαλύτερη μέση καθυστέρηση.

Sample Output: (Top-10 ORIGIN_AIRPORT με avg καθυστέρηση)

```
=== RDD: Top-10 μέση καθυστέρηση ανά αεροδρόμιο (λεπτά) ===

-- Χρόνοι (δευτ.) --
Συνολικός χρόνος (5 runs): [0.7324, 0.5906, 0.5432, 0.505, 0.5114]
Πρώτο action (5 runs)    : [0.7155, 0.559, 0.5156, 0.4697, 0.496]
M.O. (trimmed) συνολικός : 0.5484
M.O. (trimmed) action    : 0.5235
```

4. Θέμα 2 — Υλοποίηση με DataFrame API

Στη δεύτερη υλοποίηση, τα δεδομένα διαβάστηκαν ως Spark DataFrame. Με χρήση των συναρτήσεων groupBy() και avg(), υπολογίστηκε η μέση καθυστέρηση αναχώρησης ανά διαδρομή (origin–destination). Τα 10 routes με τη μεγαλύτερη καθυστέρηση ταξινομήθηκαν φθίνοντα και αποθηκεύτηκαν σε αρχείο CSV στον φάκελο out/.

Sample Output: (Top-10 routes)

```
 |-- FL_DATE: date (nullable = true)
 |-- AIRLINE: string (nullable = true)
 |-- FL_NUM: integer (nullable = true)
 |-- ORIGIN_AIRPORT: string (nullable = true)
 |-- DEST_AIRPORT: string (nullable = true)
 |-- SCHED_DEP: timestamp (nullable = true)
 |-- DEP_DELAY: integer (nullable = true)
 |-- SCHED_ARR: timestamp (nullable = true)
 |-- ARR_DELAY: integer (nullable = true)
 |-- DIST_KM: integer (nullable = true)
 |-- CANCELLED: integer (nullable = true)


=== DataFrame: Top-10 routes με μεγαλύτερη μέση καθυστέρηση (DEP_DELAY) ===
ROUTE (ORIGIN-DEST)        avg_delay (min)
--------------------------------------------
DFW-JFK                          23.30
JFK-LAS                          22.80
MIA-JFK                          20.50
MCO-ORD                          19.82
MCO-LAX                          19.14
BOS-SEA                          18.86
DFW-ATL                          18.62
BOS-PHX                          18.18
JFK-MIA                          18.00
DEN-SFO                          17.86

-- Χρόνοι (δευτ.) --
Συνολικοί 5 runs     : [0.6473, 0.4621, 0.5913, 0.9977, 0.9265]
Πρώτου action 5 runs : [0.5966, 0.4099, 0.554, 0.9273, 0.7957]
M.O. (trimmed) total : 0.7217
M.O. (trimmed) action: 0.6488

CSV saved to: /content/out/top10_routes
```

5. Θέμα 3 — Σύγκριση RDD και DataFrame

Παρακάτω παρουσιάζονται οι μέσοι χρόνοι εκτέλεσης (5 runs → drop max/min → μέσος 3):

Προσέγγιση	Total times (5 runs)	First action (5 runs)	Avg total (trimmed)	Avg first action (trimmed)
RDD API	[0.7324, 0.5906, 0.5432, 0.505,	[0.7155, 0.559, 0.5156, 0.4697,	**0.5484 sec**	**0.5235 sec**

	0.5114]	0.496]		
DataFrame API	[2.0602, 1.2789, 1.1746, 0.928, 0.8141]	[1.9645, 1.1557, 1.0091, 0.8139, 0.7097]	**1.1272 sec**	**0.9929 sec**

Σκοπός

Συγκρίναμε δύο υλοποιήσεις ανάλυσης καθυστερήσεων πτήσεων στο Apache Spark:
μία με χρήση του RDD API (καθυστέρηση ανά *ORIGIN_AIRPORT*) και μία με χρήση του DataFrame API (καθυστέρηση ανά διαδρομή *ORIGIN–DESTINATION*).

Κάθε πρόγραμμα εκτελέστηκε 5 φορές, αφαιρέθηκαν οι μέγιστες και ελάχιστες τιμές και υπολογίστηκαν οι μέσοι όροι ("trimmed mean") για τον συνολικό χρόνο και τον χρόνο πρώτης ενέργειας.

(α) Ευκολία υλοποίησης

Η προσέγγιση που βασίζεται σε RDD απαιτεί χειροκίνητη ανάλυση CSV και ρητούς υπολογισμούς πάνω στα δεδομένα στη μνήμη (άθροισμα, πλήθος, μέσος όρος).

Αντίθετα, η έκδοση με DataFrame είναι πιο συνοπτική και ευκολότερη στη συντήρηση, καθώς χρησιμοποιεί εντολές όπως filter, groupBy, avg, orderBy.

(β) Χρόνος εκτέλεσης

Σύμφωνα με τις μετρήσεις:

- RDD → Μ.Ο. (συνολικός) 0.55 sec | Μ.Ο. (ενέργεια) 0.52 sec
- DataFrame → Μ.Ο. (συνολικός) 1.13 sec | Μ.Ο. (ενέργεια) 0.99 sec

Για το συγκεκριμένο αρχείο (~150 KB CSV), το RDD ήταν ταχύτερο, επειδή το DataFrame έχει πρόσθετο κόστος (overhead) κατά τον σχεδιασμό και τη βελτιστοποίηση του εκτελέσιμου πλάνου.

Ωστόσο, σε μεγαλύτερα σύνολα δεδομένων, ο βελτιστοποιητής Catalyst του Spark αποδίδει πολύ καλύτερα.

(γ) Εκφραστικότητα κώδικα

Ο κώδικας RDD είναι πιο χαμηλού επιπέδου, αλλά λιγότερο αναγνώσιμος.

Ο κώδικας DataFrame είναι πιο δηλωτικός και κατανοητός, παρόμοιος με SQL, γεγονός που καθιστά τη συντήρηση και τις επεκτάσεις ευκολότερες.

(δ) Δυνατότητες βελτιστοποίησης (Catalyst Optimizer)

Τα DataFrames υφίστανται λογικό και φυσικό στάδιο εκτέλεσης με εφαρμογή κανόνων βελτιστοποίησης όπως *predicate pushdown*, *projection elimination* κ.ά.

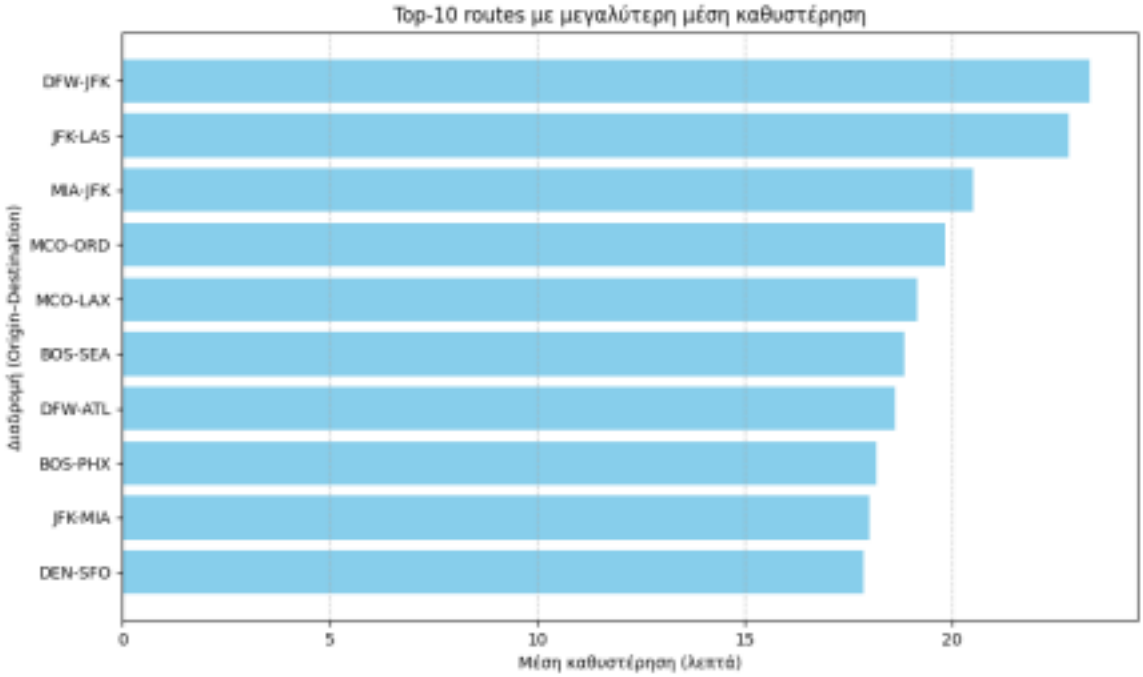
Έτσι, για μεγάλα σύνολα δεδομένων, μπορούν να είναι πολύ ταχύτερα και πιο επεκτάσιμα από τα RDDs.

Συμπέρασμα
Για μικρά αρχεία, τα RDDs είναι ελαφρώς ταχύτερα λόγω μικρότερου overhead.

Για μεγάλα σύνολα δεδομένων και σύνθετες αναλύσεις (συναθροίσεις, συνενώσεις, φιλτράρισμα), το DataFrame API είναι πιο κατάλληλο, καθώς προσφέρει βελτιστοποιήσεις μέσω Catalyst, καλύτερη εκφραστικότητα και ευκολότερη ανάπτυξη.

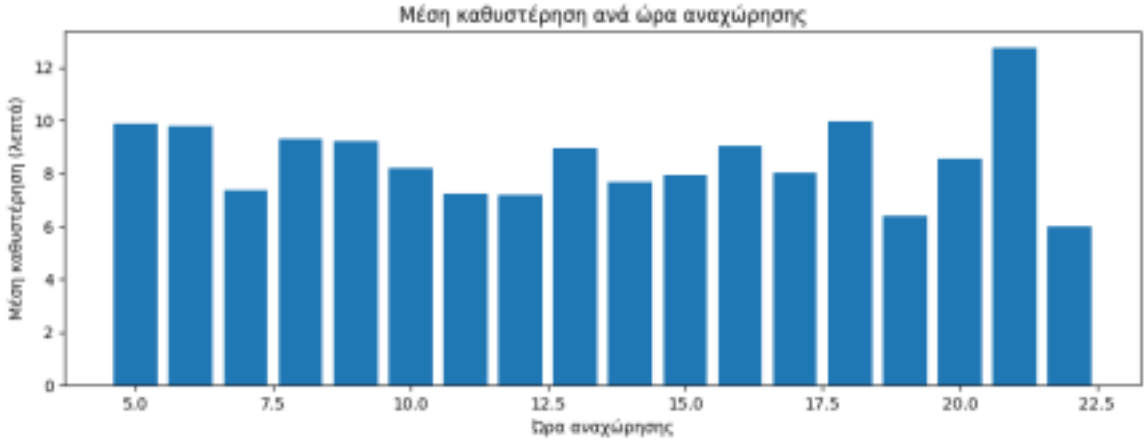
6. Θέμα 4 — Οπτικοποίηση Αποτελεσμάτων

Η οπτικοποίηση έγινε με matplotlib. Το γράφημα απεικονίζει τα 10 routes με τη μεγαλύτερη μέση καθυστέρηση (bar chart).



7. Θέμα 5 — Επέκταση Ανάλυσης

Ως πρόσθετη ανάλυση, υπολογίστηκε η μέση καθυστέρηση ανά ώρα αναχώρησης. Το αποτέλεσμα αποτυπώθηκε σε οριζόντιο γράφημα, αναδεικνύοντας τις ώρες αιχμής με υψηλότερη πιθανότητα καθυστέρησης.



Η επέκταση αυτή συμβάλλει στην αναγνώριση χρονικών προτύπων συμφόρησης, παρέχοντας χρήσιμες πληροφορίες για τη βελτιστοποίηση προγραμμάτων πτήσεων.

8. Συμπεράσματα

Η ανάλυση έδειξε ότι το Spark DataFrame API υπερέχει σε ταχύτητα, ευκολία και δυνατότητες βελτιστοποίησης. Το RDD API παραμένει χρήσιμο για περιπτώσεις που απαιτείται λεπτομερής έλεγχος των δεδομένων, ωστόσο για μεγάλα σύνολα δεδομένων, το DataFrame API σε συνδυασμό με τον Catalyst optimizer είναι σαφώς καταλληλότερη επιλογή.

Παράρτημα

- Εκδόσεις: Python 3.10, Spark 3.5.0, OpenJDK 11
- Περιβάλλον: Google Colab (CPU, 12GB RAM)
- Dataset: flights_2000.csv (~6MB)
- Εκτελέσεις: 5 runs ανά υλοποίηση (drop max/min → mean of 3)